PROJECT PROPOSAL · VERSION 1.1 · FOR FACULTY APPROVAL

# LabTrack

## Hardware Component Availability, Issuance & Tracking System

Turning fragmented laboratory hardware handling into a traceable digital lifecycle.

SY · Semester 3 · AIML 2025–29

CCE Presentation & Viva

DCDS Lab ESE Miniproject

BEFORE WE BEGIN

# What we are asking you to approve

This is a proposal, not a finished system. Every statement is labelled so that working assumptions are never mistaken for college policy.

● Observed / confirmed

Things directly observed in the current lab process, or specified in the official evaluation rubrics.

● V1 assumption — pending confirmation

Temporary working rules so development can begin. They are configurable and will be replaced by institutional answers.

● Pending faculty confirmation

Institutional decisions only faculty / college authorities can make. Collected in the decision questionnaire.

● Proposed — for approval

Our proposed technical design: architecture, data model, and technology stack. Open to technical feedback.

● Future scope

Valuable extensions deliberately deferred outside the V1 implementation boundary.

## Core principle

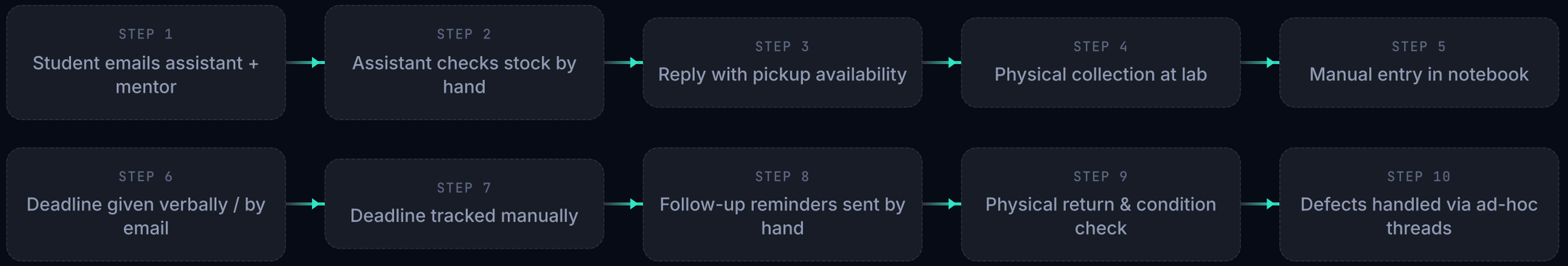
Automate the workflow lab assistants and students already perform, **without modifying departmental policy** unless explicitly authorised.

THE PROBLEM

Observed by team

# Today, lab hardware is handled through email, notebooks and manual records

The process observed for academic projects requiring hardware components:




### Stock records

Partly maintained in **Excel spreadsheets**, updated manually.

### Custody history

Logged in **physical register notebooks**.

### Queuing & demand

**No unified queue** for components that are currently issued.

CONSEQUENCES

Observed by team

# Where the current workflow creates operational friction

### Lab Assistants

- Manual availability verification
- Repetitive email inquiries
- Physical register maintenance
- Manual due date tracking
- Manual reminder drafting
- Handwritten defect & loss logs

### Students

- No live stock visibility
- Unclear which lab holds assets
- Unclear inquiry routing
- No request progress status
- No structured reservation queue
- No verifiable borrowing history

### Faculty Mentors

- Requests fragmented in inbox
- Project context disconnected
- No central approval log
- Overdue cases require manual check
- No visibility on defects or damage

### Department

- Split records (Excel vs notebooks)
- Incomplete asset traceability
- Unmet component demand invisible
- No equipment utilisation data
- Weak institutional accountability

THE PROPOSAL

# One system for the entire component lifecycle

LabTrack digitises the journey from project requirement to return, defect logging, and accountability.




Note: No performance improvements are asserted as fact. Impact will be reported only when empirically measured.

DOMAIN MODEL

# Why LabTrack is more than an inventory system

Traditional inventory answers what exists. LabTrack governs academic context, workflow authority, and physical custody.

01 · STOCK

## Inventory

**What exists?**

Item types, specifications, and total counts across department labs.

02 · CONTEXT

## Project

**Who needs it & why?**

Student team, academic purpose, and assigned faculty mentor.

03 · GOVERNANCE

## Workflow

**Who approved it?**

Configurable multi-tier faculty review and authorization.

04 · IDENTITY

## Asset

**Which exact unit?**

Specific serialized physical unit (e.g. ARDUINO-004) carrying unit history.

05 · CUSTODY

## Transaction

**Who holds it?**

Verified physical handover between assistant and student borrower.

06 · DISCIPLINE

## Accountability

**When is it due?**

Automated reminder schedule, formal extension requests, and overdue flags.

07 · TRACEABILITY

## History

**What happened?**

Inspection records, defect logs, and immutable audit trails.

TARGET ENVIRONMENT

# Sized for one department, built for portability

Approximate sizing

| 8–10 | 10–12 | 700–800 | 5–10 |
|---|---|---|---|
| Department Labs | Lab Assistants | Enrolled Students | Expected Concurrent Users |

### Prototype first

V1 is a functional prototype for demonstration and academic evaluation. Department adoption is a goal, **not** a confirmed mandate.

### Web responsive

Built as a responsive web application. Dedicated native mobile applications remain future scope.

### Infrastructure portability

Demonstrated on a cloud VM using synthetic data, designed for seamless migration to institutional servers upon approval.

WHO USES IT

# Core application roles + policy-dependent authorities

## Student

- Register project & add team members
- Select assigned faculty mentor
- Browse catalogue & real-time stock
- Submit issue / reservation requests
- Track due dates & request extensions
- View borrowing & defect history

## Faculty Mentor

- Access **authorized projects only**
- Approve, reject, or modify requests
- Receive overdue notifications
- Review extension requests
- View student custody & defect logs

## Lab Assistant

- **Physical inventory authority**
- Verify & physically issue hardware
- Assign specific serialized asset unit
- Set borrowing duration & due date
- Inspect returns & record defects
- Manage lab stock & queue overrides

## System Admin

- Manage users, roles, and lab locations
- Maintain catalogue & asset metadata
- Configure system parameters & Excel import
- Review system logs & seed demo data
- Role is **transferable**, not bound to a person

Policy authority: UAQ5

**Head of Department (HoD):** Policy-dependent authority for approval/escalation; exact system role and routing rules remain pending faculty confirmation.

DOMAIN LIFECYCLE

# One project, one traceable lifecycle

Hardware requests are bound to an authorized academic **project** rather than informal ad-hoc requests.

01
**Project**
team + mentor

02
**Requirements**
from catalogue

03
**Availability**
qty · lab · status

04
**Request**
available items

05
**Approval**
faculty review

06
**Physical Issue**
verified by assistant

**Active Issue ├─ Automated Reminders**
Scheduled notices: T-7, T-3, T-1, Due Date, and daily overdue alerts.

**Active Issue ├─ Extension Request**
Formal request → mentor approval → assistant updates transaction due date.

**Active Issue └─ FIFO Reservation**
Other students join queue for currently issued items; notified on return.

01
**Return**
physical handover

02
**Inspection**
condition verification

03
**Closure / Defect**
resolution & penalty

04
**History**
immutable audit log

■ Sequential request & issue

■ Concurrent active issue states

■ Return, inspection & audit closure

DATA MODEL

# Component Type ≠ Physical Asset

Separating catalogue definitions from individual physical hardware units.

COMPONENT TYPE

## Catalogue Definition

Arduino Uno · ESP32 · Ultrasonic Sensor HC-SR04 · Resistor · LED

Defines category, specifications, pinouts, pin compatibility, datasheet links, and catalogue alternatives.

PHYSICAL ASSET

## Serialized Unit Instance

`ARDUINO-001` · `ARDUINO-002` · `ESP32-014`

Carries current lab location, operational status, custody history, defect logs, and maintenance records.

## Individually Tracked

**Individually tracked assets carry their own physical-unit history.** Examples: Arduino, ESP32, Raspberry Pi boards.

Classification: UAQ7

## Quantity Tracked

Standard components where bulk count matters rather than individual unit history (e.g. passive sensors, discrete ICs).

## Consumable Items

Components not expected back (e.g. solder wire, discrete resistors, LEDs). Exact rules subject to UAQ7.

STATE MODEL

# Explicit separation of catalogue, asset and request states

V1 rule

Unavailable items are never submitted as normal issue requests.

## CATALOGUE / STOCK STATUS

| | |
|---|---|
| Stocked | In Catalogue |
| Not Stocked | External / Unstocked |

Determines whether an item exists in the department catalogue. If unstocked, logged as **unmet demand**.

## PHYSICAL / INVENTORY STATE

| | |
|---|---|
| Available | In Lab Storage |
| Issued | Active Custody |
| Defective / Repair | Out of Service |
| Unavailable | Decommissioned |

Reflects the actual physical condition of individual hardware units.

## REQUEST / RESERVATION STATE

| | |
|---|---|
| Fulfillable | Ready for Request |
| Reservation Queued | FIFO Order |
| Reservation Ready | Available for Pickup |
| Expired / Cancelled | Closed Queue |

**Join the FIFO queue for eligible reservation requests.**
Expiry & non-collection rules subject to UAQ11.

WORKFLOW

Routing: UAQ5

# Approval routing is configurable. Physical issue stays human.

V1 proposal: routing is stored as configuration, pending formal faculty confirmation.




## Faculty mentor can

- Approve request
- Reject request with reason
- Modify quantities / component choices

## At issue, the assistant

- Physically verifies the requested units
- Selects the specific unit (e.g. ARDUINO-007)
- Sets duration and creates the **issue transaction**

## Open policy questions

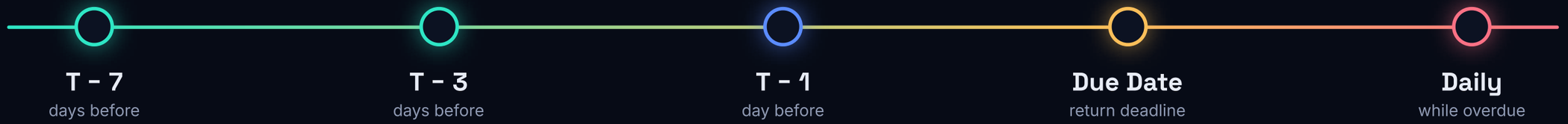
- Is HoD approval required for high-value items?
- Do multiple mentors have equal authority?
- Can an approval be revoked prior to issue?

Note: Approval routing is fully configurable in software and will adapt to confirmed departmental rules.

ACCOUNTABILITY

Timings: UAQ12 / UAQ16

# Automated reminder schedule & extension control




## Notification Channels

In-app dashboard alerts plus automated email dispatch from dedicated project address. Notices halt upon return or approved extension.

## Extension Handling

Separate auditable record: request → faculty mentor approval → assistant updates due date. Approved extensions do not incur overdue penalties.

## Overdue Escalation

Assistant monitors overdue cases; faculty review overdue alerts and decide escalation. Escalation is human-governed, not an automated punishment.

CLOSURE

# Every return is physically inspected and recorded

Student Returns Hardware → Assistant Inspects Unit → Condition Recorded → Photo Evidence (If Damaged) → Inventory State Updated → Student Acknowledged

## Defect / Damage / Loss

Records asset ID, student, project, original issue date, inspecting assistant, condition description, and optional photo. Moves asset to Defective or Under Repair. **Audit history preserved.**

## Penalties (Policy-Dependent)

Formal records for unreturned, damaged, or late items. Recorded by assistant; faculty/admin can waive or modify. **No online payment gateway** in V1.

- Fee / policy: UAQ13

## REPO: Reputation Score

**Proposed reputation score — formula and effect pending faculty confirmation (UAQ15).** Designed to be transparent, explainable, and challengeable.

- Status: Proposed Concept

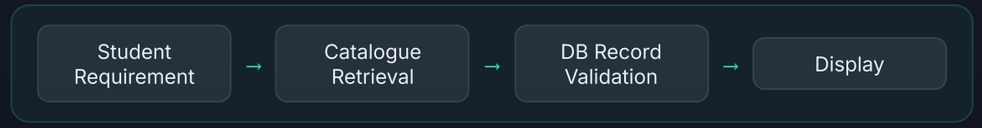
INTELLIGENCE

# AI helps students choose. The database decides.

AI is deliberately a bounded advisory layer, not the source of institutional truth.

## AI May

- Explain component functionality and pinout concepts
- Interpret student project requirements
- Suggest candidate catalogue alternatives
- Help students compare technical specifications




**AI suggestions are generated from catalogue-backed candidates and validated against current database records before being shown.**

## AI May Never

- Determine whether an item exists in stock
- Override inventory or asset availability states
- Issue hardware or approve requests
- Reorder or modify reservation queues
- Decide or calculate financial penalties
- Act as an institutional or departmental authority

**Core Guardrail:** The LLM is never the source of inventory truth.

**Unmet-Demand Analytics:** Repeated requests for unstocked components are aggregated to highlight student demand trends for departmental procurement review. Plain database aggregation — zero AI required.

ARCHITECTURE

Proposed — for approval

# Proposed technical stack: FastAPI, PostgreSQL, MongoDB, React

Clean modular architecture separating clients, service layers, and storage.

## CLIENTS & INTEGRATIONS

### Web Client (React)
Single Page Application with role-tailored views for Student, Faculty, Assistant, Admin

### Email Service
Automated notifications from dedicated system email

## EXTERNAL BOUNDARIES

### SMTP Gateway
Outbound reminders & verification codes

### LLM API Adapter
Read-only catalogue advice adapter

### Excel Importer
Bulk ingestion of existing lab inventories

## Python API — FastAPI

HTTP API · JSON · JWT Auth

### Auth & RBAC
Server-side token verification & roles

### Projects & Teams
Team members, mentors, requirements

### Catalogue & Assets
Types, units, serialized tracking

### Workflow Engine
Configurable multi-tier approval

### Issue & Return
Atomic issuance, inspection, defects

### Reservations
FIFO queue & audited assistant override

### Scheduler
Automated reminder & overdue cron jobs

### Notifications
In-app badges & email queue dispatch

### Analytics
Demand, utilisation, custody reports

### AI Advisory
Catalogue-grounded candidate advice

## DATA STORAGE

### PostgreSQL
**System of Record**

for transactional data: users, projects, requests, assets, issues, reservations, extensions, penalties.

Constraints · Foreign Keys · Stored Routines

### MongoDB
**Event & Document Store**

: audit trails, notification logs, unmet-demand events, AI logs, evidence metadata.

Aggregation Pipelines

### Deployment Target
Cloud VM with synthetic data → College server upon approval

DATA STRATEGY

Proposed — for approval

# PostgreSQL owns workflow state. MongoDB stores event records.

The split follows the fundamental difference between transactional integrity and event/document workloads.

## PostgreSQL: Transactional System of Record

- **Serialized Concurrency:** Concurrent issuance is serialized per asset using database constraints, row-level locking and transactional updates.
- **Controlled Issuance Routine:** Database routine validates availability, locks the eligible asset, creates the issue transaction and commits consistently.
- **Relational Joins:** Project → members → requests → issues → physical assets → lab locations.
- **Aggregations & Subqueries:** Overdue issues per lab, students with repeat delays.

## MongoDB: Event / Document Analytics Store

- **Polymorphic Audit Events:** Flexible event payloads (approvals, overrides, transfers, role assignments).
- **Notification History:** Delivery tracking and user notification timelines.
- **Aggregation Pipelines:** Most-requested unstocked components per month for procurement insight.
- **Turnaround Analytics:** Reservation wait-time distributions and asset utilisation rates.

**Event Consistency Boundary:** PostgreSQL remains authoritative. MongoDB records events after the authoritative transaction through reliable event delivery. MongoDB never determines business state.

SECURITY

# Proposed controls: least privilege, full audit trail, synthetic cloud data

## Access Control

- Server-side RBAC enforced on every API route
- Students see only projects they are authorized to access
- Faculty see only projects they are authorized to access
- Assistants manage only assigned department labs

## Input & Data Safety

- Strict schema validation on every payload
- Parameterized SQL and input validation to mitigate SQL injection
- Hashed passwords (bcrypt) & rate-limited auth
- Restricted defect evidence file uploads
- API responses return only necessary data fields

## Auditability

- Approvals, issues, overrides, transfers, and penalties are permanently logged
- Workflow transactions are never silently overwritten
- Admin role is transferable and auditable

## Synthetic Cloud Data

Cloud demonstration uses synthetic data only. Production handling of institutional PII is subject to college privacy/security approval (UAQ17).

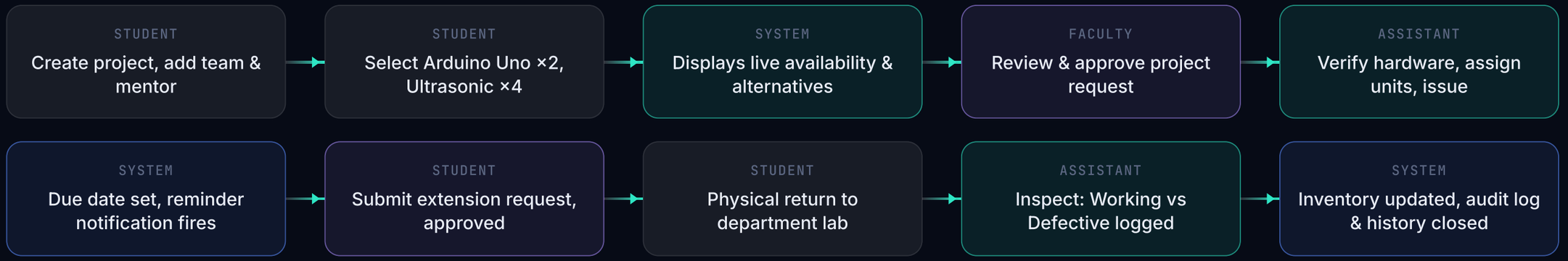
## Replaceable Authentication

V1 uses local login (PRN for students, email for staff). Architecture allows straightforward adapter replacement for College SSO (UAQ1).

LIVE DEMONSTRATION

# Planned live demonstration

End-to-end scenario to be demonstrated on the live prototype deployment:




### Exception Flow: Currently Issued

A second student sees "Currently issued" and joins the FIFO reservation queue. When returned, the first student in queue is notified.

### Exception Flow: Not Stocked

System suggests catalogue alternatives. If unavailable, logs an unmet-demand event for departmental procurement visibility.

ACADEMIC CONTEXT

CCE Literature Context

# Research context & existing literature

Situating LabTrack within existing laboratory management and resource allocation literature.

## EXISTING LITERATURE & SYSTEMS

- **Static Inventory Systems:** Focus primarily on stock levels and reorder thresholds without academic project context.
- **General Resource Reservation:** Manage room bookings or library assets, lacking multi-week hardware lifecycle and defect tracking.
- **RFID / IoT Tracking:** Focus on hardware localization without integrating multi-tier faculty approval or student team accountability.

## LABTRACK'S INTEGRATED FOCUS

- **Project-Bound Lifecycle:** Hardware custody tied to validated academic projects, teams, and faculty mentors.
- **Multi-Tier Workflow Authority:** Configurable faculty review, HoD governance, and physical assistant verification.
- **Serialized Concurrency & FIFO:** Race-condition-free asset issuance with audited reservation queues.
- **Demand & Accountability Analytics:** Unmet demand logging to inform department procurement decisions.

**Synthesis:** Rather than asserting an absence of prior work, LabTrack synthesizes inventory tracking, workflow approval, automated reminders, and audit trails into a single unified academic laboratory workflow.

ACADEMIC RELEVANCE

# How LabTrack will address the rubric

Planned coverage mapped directly to official evaluation guidelines.

## DCDS LAB ESE · 10 MARKS · 40% WEIGHTAGE

| RUBRIC CRITERION | MARKS | PLANNED V1 IMPLEMENTATION |
|---|---|---|
| Application Design & Functionality | 2 | Role-based dashboards, full lifecycle, modular service architecture |
| Python Programming Implementation | 2 | OOP classes, exception handling, modular packages, Excel ingestion |
| Database Integration (SQL & NoSQL) | 2 | PostgreSQL relational schema + MongoDB event & audit store |
| Advanced Database Querying & Retrieval | 2 | Complex joins, subqueries, grouping, stored routines, Mongo aggregations |
| Code Quality, Documentation & UI | 1 | Clean UI, structured repository, comprehensive markdown context docs |
| Testing, Validation, Security & Errors | 1 | Parameterized queries, server-side RBAC, validation schemas, unit tests |

## CCE EVALUATION · 30 MARKS · CO3 / CO4

| COMPONENT | MARKS | COVERAGE |
|---|---|---|
| Topic Submission | 5 | Unified CCE + DCDS Proposal |
| Research / Literature Survey | 5 | Literature context & gap analysis |
| Presentation Skills | 5 | Structured slide deck & architecture |
| Running Example (Live Cloud) | 10 | Full cloud prototype demonstration |
| Viva Voce | 5 | Technical defense & Q&A |

TIMELINE

Proposed plan

# 4-Week delivery plan & feature prioritization

Working backwards from the DCDS Lab ESE schedule.

WEEK 1

## Foundation

- Faculty confirmation on P0 rules
- Schema & API specifications
- DB migrations, Auth & RBAC
- Catalogue & Excel importer

WEEK 2

## Core Lifecycle

- Project & team management
- Request → approval workflow
- Controlled issuance routine
- Role-tailored dashboards

WEEK 3

## Accountability

- Reminder & extension engine
- FIFO reservation queue
- Return & defect inspection
- Audit trails & Mongo pipelines

WEEK 4

## Harden & Ship

- AI advisory adapter
- Testing, security & validation
- Cloud prototype deployment
- Documentation & viva prep

**Scope Discipline (Triage Order):** **Must-Have:** Auth, RBAC, DB schemas, Catalogue, Projects, Requests, Approval, Issue/Return, PostgreSQL, MongoDB, Advanced queries, Security, Cloud VM. **Should-Have:** FIFO Reservations, Reminders, Extensions, Defect handling. **Cut First If Time Runs Short:** REPO scoring, AI advisory layer, Asset transfers, advanced analytics. The core lifecycle and DCDS database requirements are never cut.

SCOPE DISCIPLINE

# Deliberately out of scope for V1

Future scope

Strict scope boundaries to ensure timely delivery and rock-solid software quality.

- Native mobile apps / Play Store release
- Cabinet / shelf / drawer sensor bins
- Full procurement purchasing workflow
- Payment gateway & fine collection
- Formal dispute / arbitration tribunals
- Enterprise accounting & depreciation
- Full college SSO (until confirmed)
- Cross-department asset sharing
- Manufacturer warranty claims tracking
- Heavy predictive ML models
- Physical register legacy OCR migration
- Real student PII on external servers

**Architectural Extensibility:** V1 is intentionally bounded. The core model is designed to remain extensible, although some future features may require additional schema or workflow changes.

DECISIONS WE NEED FROM YOU

# High-priority institutional policy questions (P0)

P0 Decisions

Key decisions required to finalize database constraints and workflow logic.

UAQ5 · APPROVAL ROUTE

### Approval Hierarchy

Is HoD approval required for specific assets? Do co-mentors have equal authority? Can approvals be revoked?

UAQ12 · BORROWING

### Duration & Extensions

Standard borrowing period (e.g. 14 days)? Maximum allowed extensions? Who approves extensions?

UAQ13 · PENALTIES

### Late / Damage Policy

Are financial penalties levied, or purely academic blocks? Who has authority to waive or reduce penalties?

UAQ16 · REMINDERS

### Reminder Schedule

Proposed T-7, T-3, T-1 schedule acceptable? At what overdue threshold should mentors/HoD be notified?

UAQ7 · TRACKING

### Item Classification

Which items are individually tracked (serialized) vs quantity-tracked vs non-returnable consumables?

UAQ8 · ASSET IDS

### Existing Identifiers

Do existing lab registers use legacy barcodes or serial tags that LabTrack must preserve?

UAQ1 · AUTHENTICATION

### Identity & Master Data

Can LabTrack integrate with college directory services, or utilize local PRN/email credentials for V1?

UAQ17/18 · CLOUD & PRIVACY

### Hosting & Privacy

Cloud demonstration with synthetic data acceptable? Requirements for future internal server deployment?

Note: Complete 22-question questionnaire (UAQ1–UAQ22) is documented in the master project context.

APPROVAL REQUEST

# What we need today

### 1 · Confirm the Project Topic

LabTrack as our unified project for both Cloud Computing Essentials (CCE) and DCDS Lab ESE.

### 2 · Validate the Workflow

Review and validate the proposed lifecycle: approval routing, borrowing limits, return inspection, and escalation rules.

### 3 · Confirm Institutional Constraints

Provide guidance on authentication, student data privacy, cloud demonstration, and eventual campus deployment.

### 4 · Connect Us with One Lab Assistant

Facilitate a short consultation with a lab assistant to review existing register formats and inventory classification.

**Proposed Technical Stack:** FastAPI (Python) + PostgreSQL + MongoDB + React SPA

Open for Technical Feedback

LabTrack turns fragmented laboratory hardware handling into a traceable digital lifecycle.

# Thank you.

Questions, suggestions & feedback welcome.

- Observed
- V1 Assumption
- Pending Confirmation
- Proposed Design
- Future Scope